10 Boolean logic

In this chapter, you will learn about:

- ★ the identification, definition, symbols and functions of the standard logic gates: NOT, AND, OR, NAND, NOR and XOR
- ★ how to use logic gates to create logic circuits from:
  - a given problem
  - a logic expression
  - a truth table
- ★ how to complete truth tables from:
  - a given problem
  - a logic expression
  - a logic circuit
- ★ how to write a logic expression from:
  - a given problem
  - a logic circuit
  - a truth table.

10.1 Standard logic gate symbols

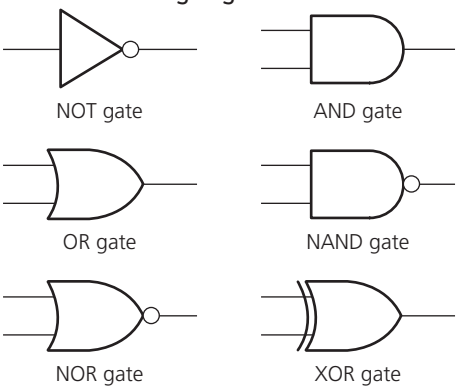
Electronic circuits in computers, solid state drives and controlling devices are made up of thousands of **logic gates**. Logic gates take binary inputs and produce a binary output. Several logic gates combined together form a **logic circuit** and these circuits are designed to carry out a specific function.

The checking of the output from a logic gate or logic circuit is done using a **truth table**.

This chapter will consider the function and role of logic gates, logic circuits and truth tables. Also a number of possible applications of logic circuits will be considered. A reference to **Boolean algebra** will be made throughout the chapter; but this is really outside the scope of this text book. However, Boolean algebra will be seen on many logic gate websites and is included here for completeness, since many students may prefer this notation to logic statements.

10.1.1 Logic gate symbols

Six different logic gates will be considered in this chapter:



▲ **Figure 10.1** Logic gate symbols

Truth tables

Truth tables are used to trace the output from a logic gate or logic circuit. The NOT gate is the only logic gate with one input; the other five gates have two inputs (see Figure 10.1).

Although each logic gate can only have one or two inputs, the number of inputs to a logic circuit can be more than 2; for example, three inputs give a possible 2^3 (=8) binary combinations. And for four inputs, the number of possible binary combinations is 2^4 (=16). It is clear that the number of possible binary combinations is a multiple of the number 2 in every case. The possible inputs in a truth table can be summarised as shown in Table 10.1.

▼ **Table 10.1** All possible inputs for truth tables with two, three and four inputs

Inputs	
A	B
0	0
0	1
1	0
1	1

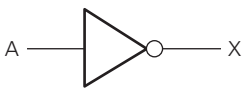
Inputs		
A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Inputs			
A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

As we can see, a truth table will also list the output for every possible combination of inputs.

10.2 The function of the six logic gates

10.2.1 NOT gate



▲ **Figure 10.2**

Description:	Truth table:	How to write this:
The output, X, is 1 if: the input, A, is 0	▼ **Table 10.2** **Input: A** / **Output: X** 0 / 1 1 / 0	X = NOT A (logic notation) X = $\overline{A}$ (Boolean algebra)

Note the use of Boolean algebra to represent logic gates. This is optional at IGCSE but many students may prefer to use this notation (see NOTE later).

10.2.2 AND gate



▲ **Figure 10.3**

Description:	Truth table:	How to write this:
The output, X, is 1 if: both inputs, A and B, are 1	▼ **Table 10.3** **Inputs: A, B** / **Outputs: X** 0 0 0 0 1 0 1 0 0 1 1 1	X = A AND B (logic notation) X = A . B (Boolean algebra)

10.2.3 OR gate



▲ **Figure 10.4**

Description:	Truth table:	How to write this:
The output, X, is 1 if: either input, A or B, or both, are 1	▼ **Table 10.4** **Inputs: A, B** / **Output: X** 0 0 0 0 1 1 1 0 1 1 1 1	X = A OR B (logic notation) X = A + B (Boolean algebra)

10.2.4 NAND gate (NOT AND)



▲ **Figure 10.5**

Description:	Truth table:	How to write this:
The output, X, is 1 if: input A AND input B are NOT both 1	▼ **Table 10.5** **Inputs** / **Output** **A** **B** **X** 0 0 1 0 1 1 1 0 1 1 1 0	X = A NAND B (logic notation) $X = \overline{A \cdot B}$ (Boolean algebra)

10.2.5 NOR gate (NOT OR)



▲ **Figure 10.6**

Description:	Truth table:	How to write this:
The output, X, is 1 if: neither input A nor input B is 1	▼ **Table 10.6** **Inputs** / **Output** **A** **B** **X** 0 0 1 0 1 0 1 0 0 1 1 0	X = A NOR B (logic notation) $X = \overline{A + B}$ (Boolean algebra)

10.2.6 XOR gate



▲ **Figure 10.7**

Description:	Truth table:	How to write this:
The output, X, is 1 if: (input A is 1 AND input B is 0) **or** (input A is 0 AND input B is 1)	▼ **Table 10.7** **Inputs** / **Output** **A** **B** **X** 0 0 0 0 1 1 1 0 1 1 1 0	X = A XOR B (logic notation) $X = (A \cdot \overline{B}) + (\overline{A} \cdot B)$ (Boolean algebra) NOTE: this is sometimes written as: $(A + B) \cdot \overline{(A \cdot B)}$

Activity 10.1

Show why X = (A AND NOT B) OR (NOT A AND B)
and
Y = (A OR B) AND (NOT (A AND B)) both represent the same logic gate.

You will notice in the Boolean algebra, three new symbols; these have the following meaning:

- . represents the AND operation
- + represents the OR operation
- a bar (above the letter or letters, e.g. $\overline{a}$) represents the NOT operation.

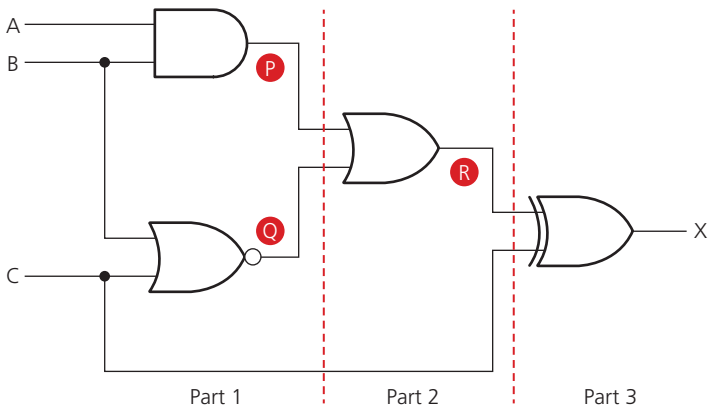
10.3 Logic circuits, logic expressions, truth tables and problem statements

When logic gates are combined together to carry out a particular function, such as controlling a robot, they form a logic circuit. The following eight examples show how to carry out the following tasks:

- Create a logic circuit from a:
  - problem statement (examples 6 and 7)
  - logic or Boolean expression (examples 3 and 8)
  - truth table (examples 4 and 5)
- Complete a truth table from a:
  - problem statement (examples 6 and 7)
  - logic or Boolean expression (examples 3 and 8)
  - logic circuit (example 1)
- Write a logic or Boolean expression from a:
  - problem statement (examples 6 and 7)
  - logic circuit (example 2)
  - truth table (examples 4 and 5).

Example 1

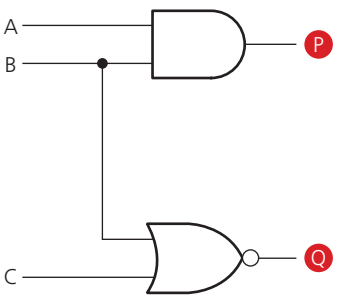
Produce a truth table for the following logic circuit (note the use of black circles at the junctions between wires):



▲ **Figure 10.8**

There are three inputs to this logic circuit, therefore, there will be eight possible binary values that can be input.

To show stepwise how the truth table is produced, the logic circuit has been split up into three parts, as shown by the dotted lines, and intermediate values are shown as P, Q and R.



▲ **Figure 10.9**

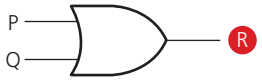
Part 1

This is the first part of the logic circuit; the first task is to find the intermediate values P and Q.

The value of P is found from the AND gate where the inputs are A and B. The value of Q is found from the NOR gate where the inputs are B and C. An intermediate truth table is produced using the logic function descriptions in Section 10.2.

▼ **Table 10.8**

input values			Output values	
A	**B**	**C**	**P**	**Q**
0	0	0	**0**	**1**
0	0	1	**0**	**0**
0	1	0	**0**	**0**
0	1	1	**0**	**0**
1	0	0	**0**	**1**
1	0	1	**0**	**0**
1	1	0	**1**	**0**
1	1	1	**1**	**0**



▲ **Figure 10.10**

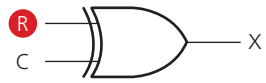
Part 2

The second part of the logic circuit has P and Q as inputs and the intermediate output, R:

This produces the following intermediate truth table. (Note: even though there are only two inputs to the logic gate, we have generated eight binary values in part 1 and these must all be used in this second truth table).

▼ **Table 10.9**

Inputs		Output
P	**Q**	**R**
0	1	**1**
0	0	**0**
0	0	**0**
0	0	**0**
0	1	**1**
0	0	**0**
1	0	**1**
1	0	**1**



▲ **Figure 10.11**

Part 3

The final part of the logic circuit has R and C as inputs and the final output, X:

This gives the third intermediate truth table:

▼ **Table 10.10**

Inputs		Output
R	**C**	**X**
1	0	**1**
0	1	**1**
0	0	**0**
0	1	**1**
1	0	**1**
0	1	**1**
1	0	**1**
1	1	**0**

Putting all three intermediate truth tables together produces the final truth table, which represents the original logic circuit:

▼ **Table 10.11**

Input values			Intermediate values			Output
A	**B**	**C**	**P**	**Q**	**R**	**X**
0	0	0	*0*	*1*	*1*	1
0	0	1	*0*	*0*	*0*	1
0	1	0	*0*	*0*	*0*	0
0	1	1	*0*	*0*	*0*	1
1	0	0	*0*	*1*	*1*	1
1	0	1	*0*	*0*	*0*	1
1	1	0	*1*	*0*	*1*	1
1	1	1	*1*	*0*	*1*	0

The intermediate values can be left out of the final truth table, but it is good practice to leave them in until you become confident about producing the truth tables. The final truth table would then look like this:

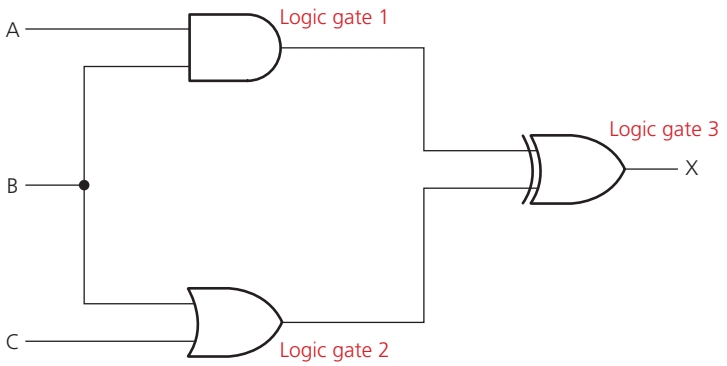
▼ **Table 10.12**

Input values			Output
A	**B**	**C**	**X**
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Example 2

Write logic expressions from the following logic circuits:

a



▲ **Figure 10.12**

The first action is to look at the gates connected to the inputs A, B and C:

logic gate 1: (A AND B)

logic gate 2: (B OR C)

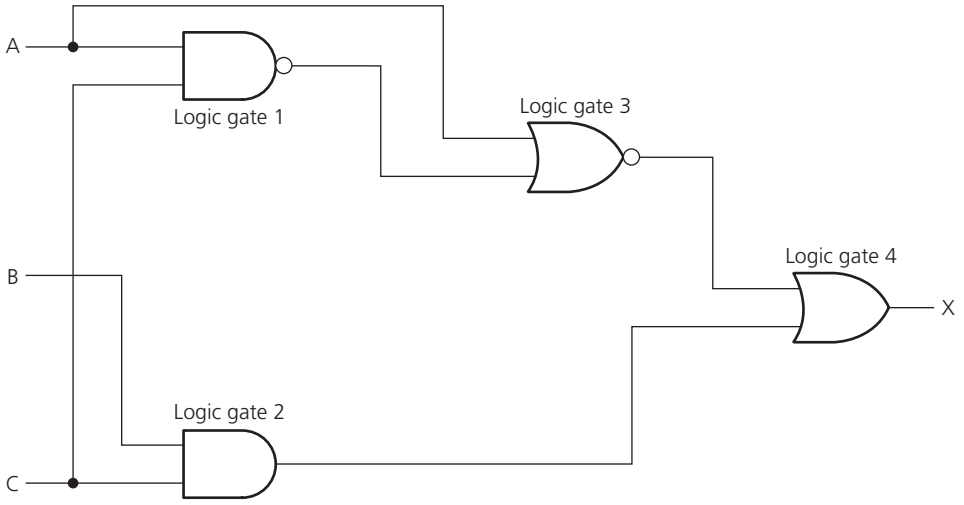
We then join these together using logic gate 3:

[(A AND B)] XOR [(B OR C)] which gives us the required logic expression.

(Note: the square brackets "[ ]" in the expression are not necessary and are used here just for clarity.)

This would be written as: (A AND B) XOR (B OR C)

b



▲ **Figure 10.13**

Again, we will do this in the order of logic gates 1 and 2 first (connected to the three inputs):

logic gate 1: (A NAND C)

logic gate 2: (B AND C)

However, logic gate 3 is also connected to one of the inputs so that should be done next:

logic gate 3: (logic gate 1) NOR A

If we replace (logic gate 1) by the logic expression above, we get:

((A NAND C) NOR A)

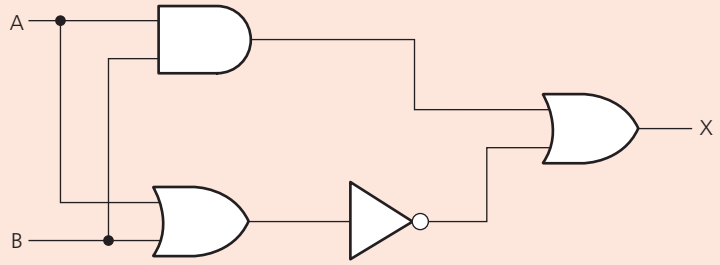
Finally, we can join all these together using:

logic gate 4: ((A NAND C) NOR A) OR (B AND C)

Activity 10.2

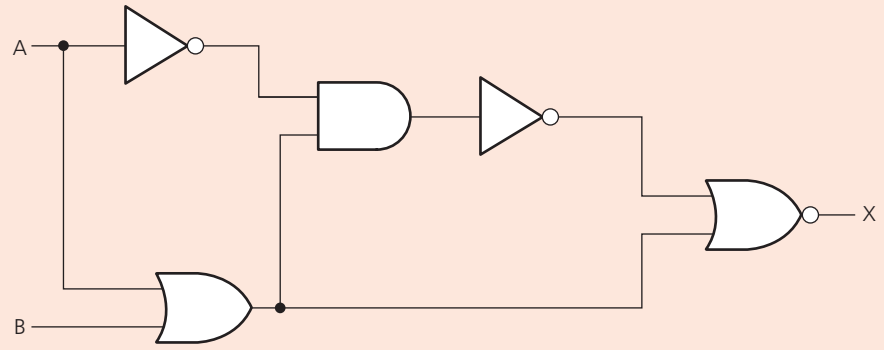
1 Produce truth tables from the following logic circuits:

a



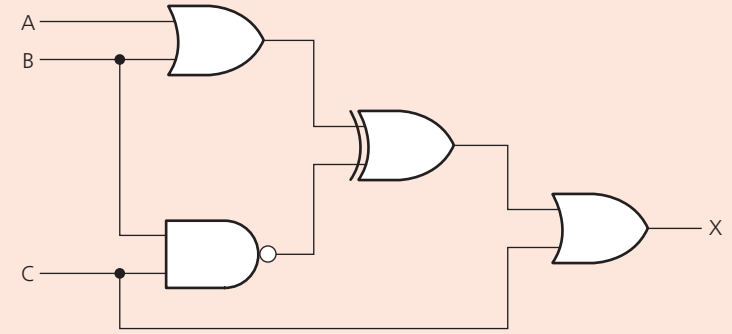
▲ **Figure 10.14**

b



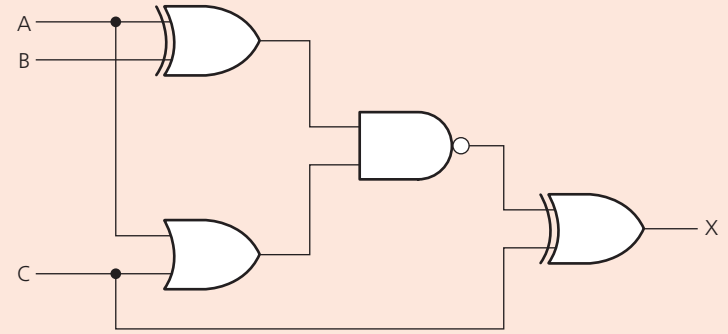
▲ **Figure 10.15**

c



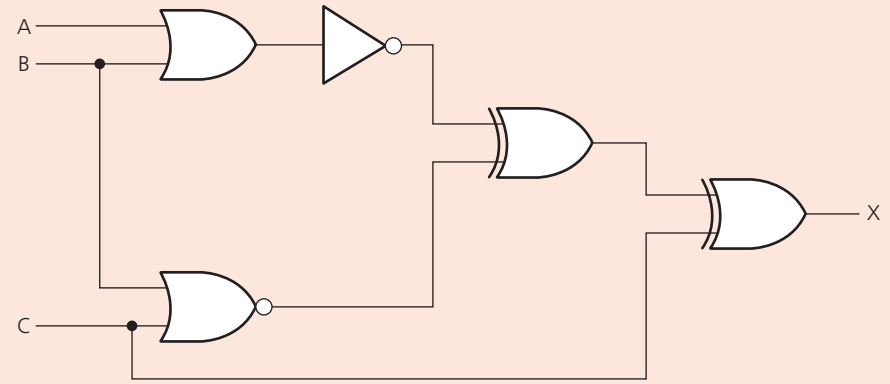
▲ **Figure 10.16**

d



▲ **Figure 10.17**

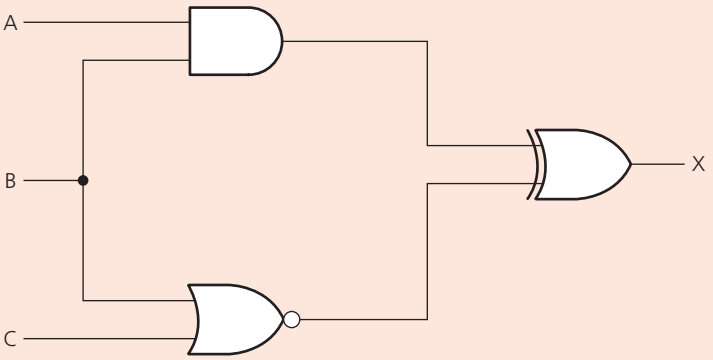
e



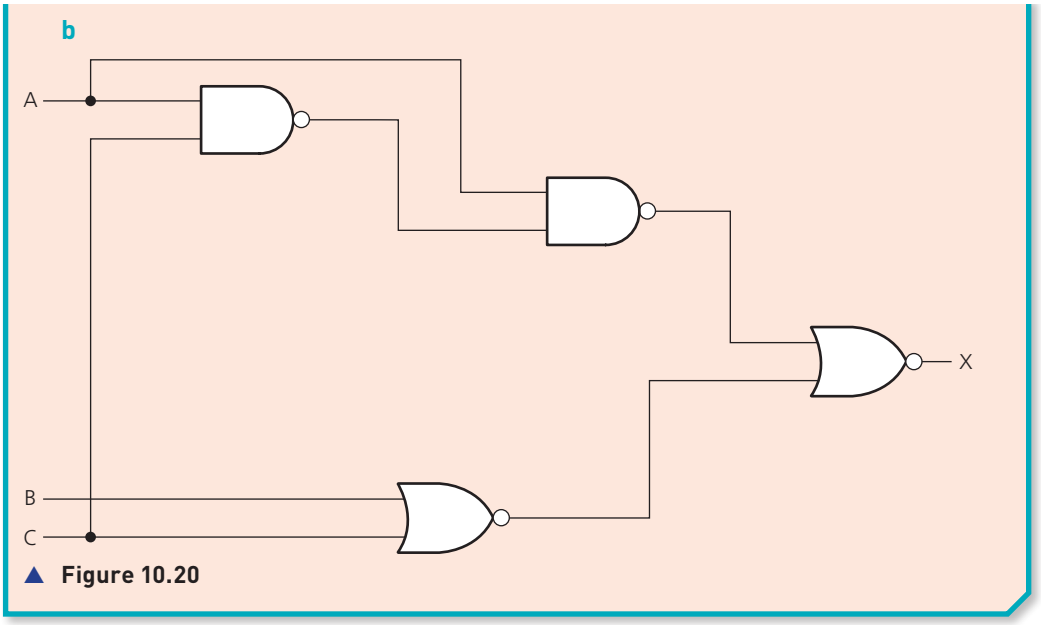
▲ **Figure 10.18**

2 Write logic expressions for the following logic circuits:

a



▲ **Figure 10.19**



▲ **Figure 10.20**

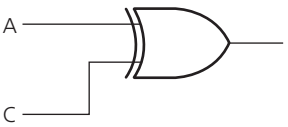
Example 3

A logic circuit can be represented by the following logic expression: (A XOR C) OR (NOT C NAND B)

Produce a logic circuit and a truth table from the above statement.

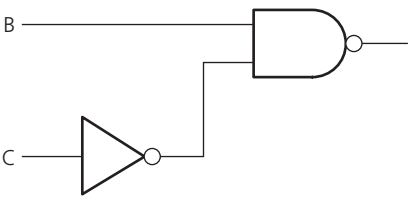
In this example we have a connecting logic gate which is OR.

So, if we produce one half of the circuit from (A XOR C) we get:



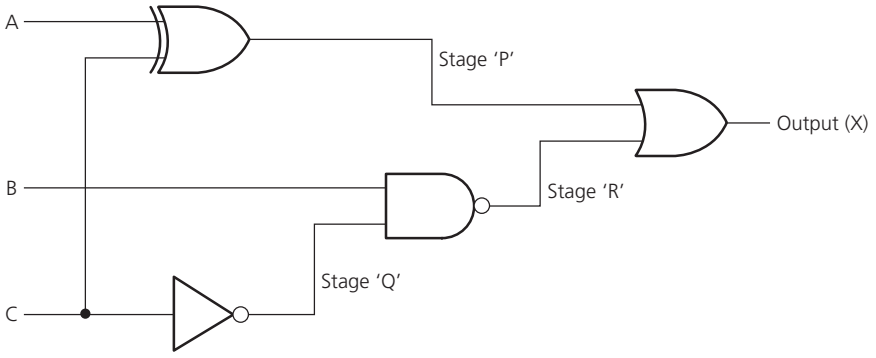
▲ **Figure 10.21**

The other half of the circuit is found from (NOT C NAND B):



▲ **Figure 10.22**

If we now combine these together to form the final logic circuit:



▲ **Figure 10.23**

The truth table is shown:

▼ **Table 10.13**

Input values			Values at stages:			Output
A	**B**	**C**	**'P'**	**'Q'**	**'R'**	**X**
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	0	1	0	0
0	1	1	1	0	1	1
1	0	0	1	1	1	1
1	0	1	0	0	1	1
1	1	0	1	1	0	1
1	1	1	0	0	1	1

Example 4

Look at the two truth tables below; in each case produce a logic expression and the corresponding logic circuit:

a

▼ **Table 10.14**

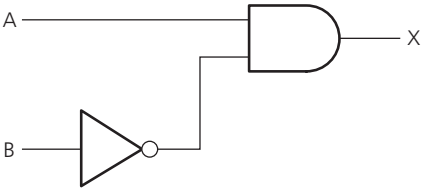
Inputs		Output
A	**B**	**X**
0	0	0
0	1	0
1	0	1
1	1	0

To produce the logic statement, we only concern ourselves with the truth table row where the output value is 1. In this case, A = 1 and B = 0 which gives the logical expression:

A AND NOT B

So we have the logic expression: A AND NOT B

(Note that this could be written as $A \cdot \overline{B}$ in Boolean.) It is now possible to draw the corresponding logic circuit:



▲ **Figure 10.24**

b

▼ **Table 10.15**

Inputs		Output
A	**B**	**X**
0	0	0
0	1	1
1	0	0
1	1	1

This time we have **two** rows where the output is 1; this gives the following logical expressions:

(NOT A AND B)

(A AND B)

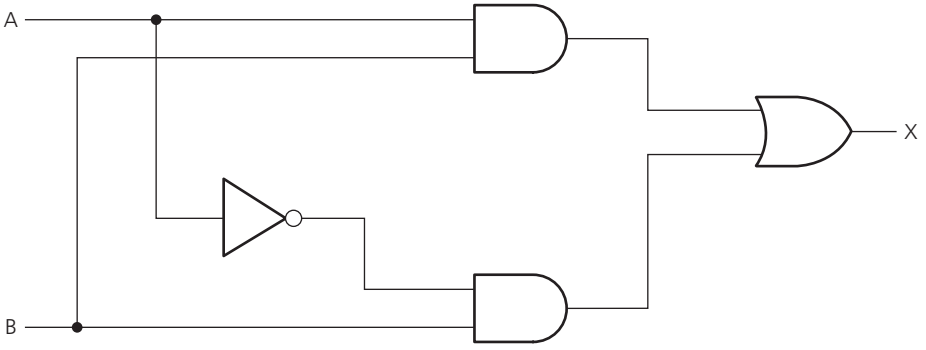
We now join these together using an OR gate to give:

(NOT A AND B) OR (A AND B)

So we have the logic expression: (NOT A AND B) OR (A AND B)

(This could be written as: $\overline{A}.B + A.B$)

It is now possible to draw the corresponding logic circuit:



▲ **Figure 10.25**

Example 5

a Which Boolean expression is represented by the following truth table?

▼ **Table 10.16**

Input values			Output
A	**B**	**C**	**X**
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

We only need to consider those rows where the output is a 1. This gives us the following three logic expressions:

(NOT A AND NOT B AND NOT C)
(A AND NOT B AND NOT C)
(A AND B AND NOT C)

If we now join the three expressions with an OR gate, we end up with the final logic expression:

(NOT A AND NOT B AND NOT C) OR (A AND NOT B AND NOT C) OR (A AND B AND NOT C)

b i Which logic expression is represented by the following truth table?
ii Show that your logic expression in part **i** is the same as: (B AND C) OR (A AND C) OR (A AND B)

▼ **Table 10.17**

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

i We only need to consider those rows where the output is a 1. This gives us the following four Boolean expressions:

(NOT A AND B AND C)
(A AND NOT B AND C)
(A AND B AND NOT C)
(A AND B AND C)

If we now join the four expressions with an OR gate we end up with the following logic expression:

(NOT A AND B AND C) OR (A AND NOT B AND C) OR (A AND B AND NOT C) OR (A AND B AND C)

ii To show that **(B AND C) OR (A AND C) OR (A AND B)** produces the same output as that shown in part **i** we need to produce a new truth table and show that the output is the same as the one in the given truth table:

▼ **Table 10.18**

A	B	C	B AND C	A AND C	A AND B	(B AND C) OR (A AND C) OR (A AND B)	X
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	1	0	1	0	1	1
1	1	0	0	0	1	1	1
1	1	1	1	1	1	1	1

As the second truth table shows, the outputs from logic expressions are both the same; thus the logic expression **(B AND C) OR (A AND C) OR (A AND B)** gives the same output as the logic expression in part **i**.

Activity 10.3

1 Produce:

i a truth table

ii a logic circuit

from the following logic expression:

(NOT A AND B) AND (NOT B OR C)

2 Produce:

i a truth table

ii a logic circuit

from the following logic expression:

(A XOR B) OR ((B NOR C) AND B)

3 Produce:

i a logic expression

ii a logic circuit

from the following truth table:

A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

4 Write down a logic expression for **each** of the following truth tables:

a

A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

b

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

5 **a** Write down a logic expression corresponding to the following truth table:

A	B	C	X
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

b Show that the following logic expression produces the same output as your answer to part **a** above:

(NOT A AND NOT B) OR (A AND NOT B)

Example 6

A safety system uses three inputs to a logic circuit. An alarm, X, sounds if input A represents ON and input B represents OFF; or if input B represents ON and input C represents OFF.

Produce a logic circuit and truth table to show the conditions that cause the output X to be 1.

The first thing to do is to write down the logic statement representing the scenario in this example. To do this, it is necessary to recall that ON = 1 and OFF = 0 and also that 0 is usually considered to be NOT 1.

So we get the following logic expression:

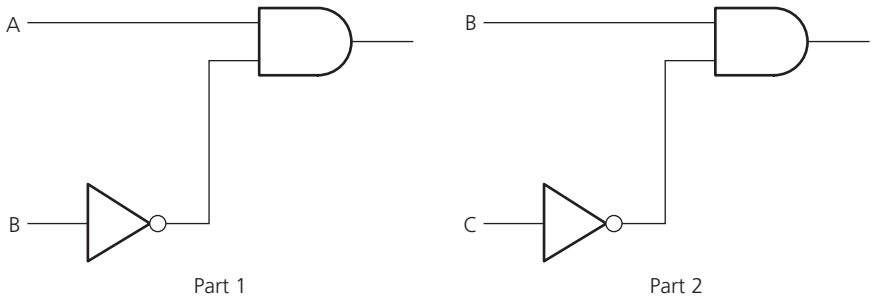
(A AND NOT B) OR (B AND NOT C)

- (A AND NOT B): this equates to: A is ON and B is OFF
- OR: the two parts are connected by the OR gate
- (B AND NOT C): this equates to B is ON AND C is OFF

Note: this expression can also be written as follows (Boolean expression):

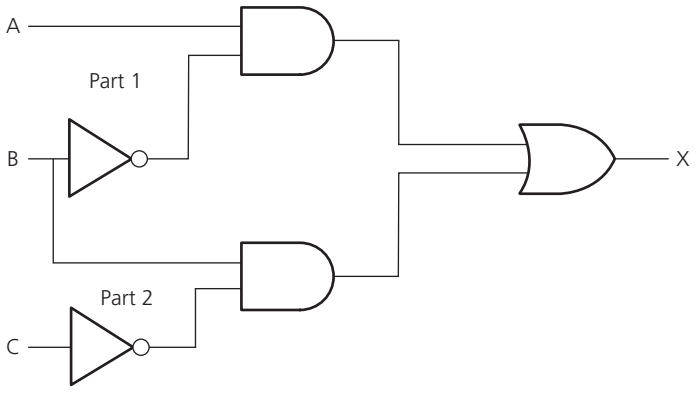
$$(A \,.\, \overline{B}) + (B \,.\, \overline{C})$$

The logic circuit is made up of two parts as shown in the logic expression. We will produce the logic gate for the first second part. Then join both parts together with the OR gate.



▲ **Figure 10.26**

Now combining both parts with the OR gate gives us:



▲ **Figure 10.27**

In order to produce the truth table, there are two ways to do this:

- trace through the logic circuit using the method described in example 1 (Section 10.3)
- produce the truth table using the original logic expression; this second method has the advantage that it allows you to check that your logic circuit is correct.

We will use the second method in this example:

▼ **Table 10.19**

Inputs			Intermediate values		Output
A	B	C	(A AND NOT B)	(B AND NOT C)	X
0	0	0	*0*	*0*	0
0	0	1	*0*	*0*	0
0	1	0	*0*	*1*	1
0	1	1	*0*	*0*	0
1	0	0	*1*	*0*	1
1	0	1	*1*	*0*	1
1	1	0	*0*	*1*	1
1	1	1	*0*	*0*	0

(Note: it is optional whether to leave in the intermediate values or simply remove them giving a 4-column truth table with headings: A, B, C, X)

? Example 7

A wind turbine has a safety system which uses three inputs to a logic circuit. A certain combination of conditions results in an output, X, from the logic circuit being equal to 1. When the value of X = 1 then the wind turbine is shut down.

The following table shows which parameters are being monitored and form the three inputs to the logic circuit.

▼ **Table 10.20**

parameter description	parameter	binary value	description of condition
turbine speed	S	0	turbine speed <= 1000 rpm
		1	turbine speed > 1000 rpm
bearing temperature	T	0	bearing temperature <= 80°C
		1	bearing temperature > 80°C
wind velocity	W	0	wind velocity <= 120 kph
		1	wind velocity >120 kph

The output, X, will have a value of 1 if any of the following combination of conditions occur:

Either turbine speed <= 1000 rpm and bearing temperature > 80°C

Or turbine speed > 1000 rpm and wind velocity > 120 kph

Or bearing temperature <= 80°C and wind velocity > 120 kph

Design the logic circuit and complete the truth table to produce a value of X = 1 when either of the three conditions above occur.

In this example, a real situation is given and it is necessary to convert the information into a logic expression and then produce the logic circuit and truth table. It is advisable in problems as complex as this to produce the logic circuit and truth table separately (based on the conditions given) and then check them against each other to see if there are any errors.

Stage 1

The first thing to do is to convert each of the three statements into logic expressions. Use the information given in the table and the three condition statements to find how the three parameters S, T and W are linked. We usually look for the key words AND, OR and NOT when converting actual statements into logic.

We end up with the following three logic expressions:

i turbine speed <= 1000 rpm and bearing temperature > 80°C
logic expression: (NOT S AND T)

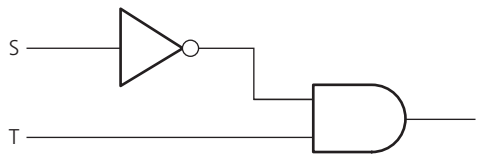
ii turbine speed > 1000 rpm and wind velocity > 120 kph
logic expression: (S AND W)

iii bearing temperature <= 80°C and wind velocity > 120 kph
logic expression: (NOT T AND W)

Stage 2

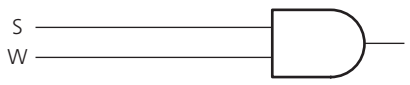
This now produces three intermediate logic circuits:

i



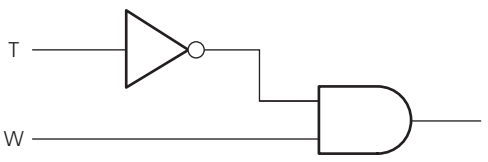
▲ **Figure 10.28**

ii



▲ **Figure 10.29**

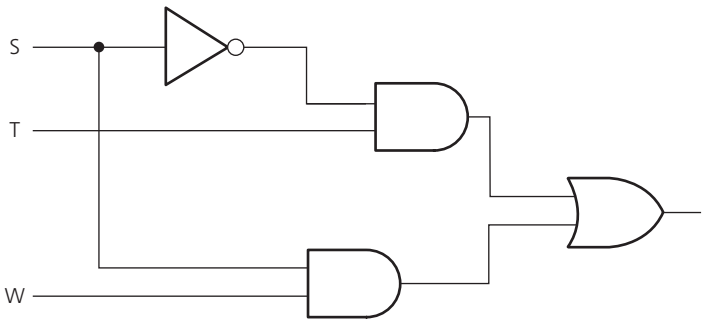
iii



▲ **Figure 10.30**

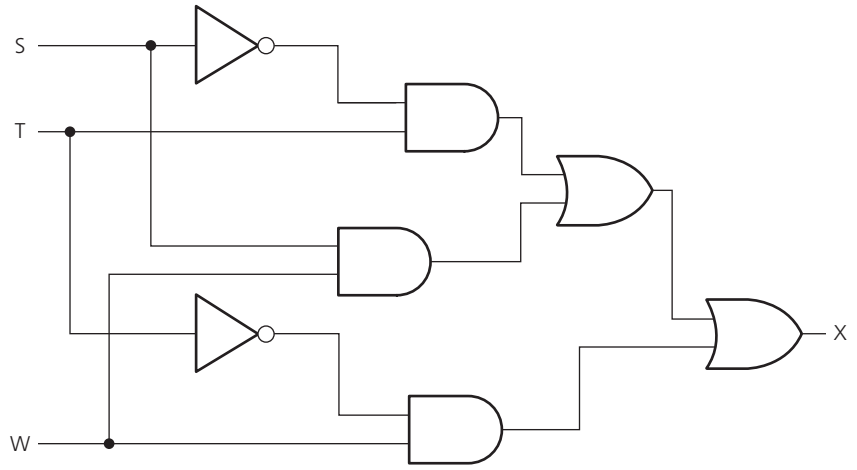
Each of the three original statements were joined together by the word 'OR'. Thus, we need to join all of the three intermediate logic circuits by two OR gates to get the final logic circuit.

We will start by joining i and ii together using an OR gate:



▲ **Figure 10.31**

Finally, we connect the logic circuit in Figure 10.31 to Figure 10.30 to obtain the answer:



▲ **Figure 10.32**

The final part is to produce the truth table. We will do this using the original logic statement. This method has the bonus of allowing an extra check to be made on the logic circuit in Figure 10.32 to see whether or not it is correct. It is possible, however, to produce the truth table straight from the logic circuit in Figure 10.32.

There were three parts to the problem, so the truth table will first evaluate each part. Then, by applying OR gates, as shown below, the final value, X, is obtained:

i (NOT S AND T)
ii (S AND W)
iii (NOT T AND W)

We find the outputs from parts (i) and (ii) and then OR these two outputs together to obtain a new intermediate, which we will now label part (iv).

We then OR parts (iii) and (iv) together to get the value of X.

Inputs			Intermediate values				Output
S	**T**	**W**	**(i) (NOT S AND T)**	**(ii) (S AND W)**	**(iii) (NOT T AND W)**	**(iv)**	**X**
0	0	0	*0*	*0*	*0*	*0*	0
0	0	1	*0*	*0*	*1*	*0*	1
0	1	0	*1*	*0*	*0*	*1*	1
0	1	1	*1*	*0*	*0*	*1*	1
1	0	0	*0*	*0*	*0*	*0*	0
1	0	1	*0*	*1*	*1*	*1*	1
1	1	0	*0*	*0*	*0*	*0*	0
1	1	1	*0*	*1*	*0*	*1*	1

Example 8

Consider the logic statement:

((A NOR B) AND C) NAND (A OR NOT B)

a Draw a logic circuit to represent the given logic statement.
b Complete the truth table for the given logic statement.

▼ **Table 10.21**

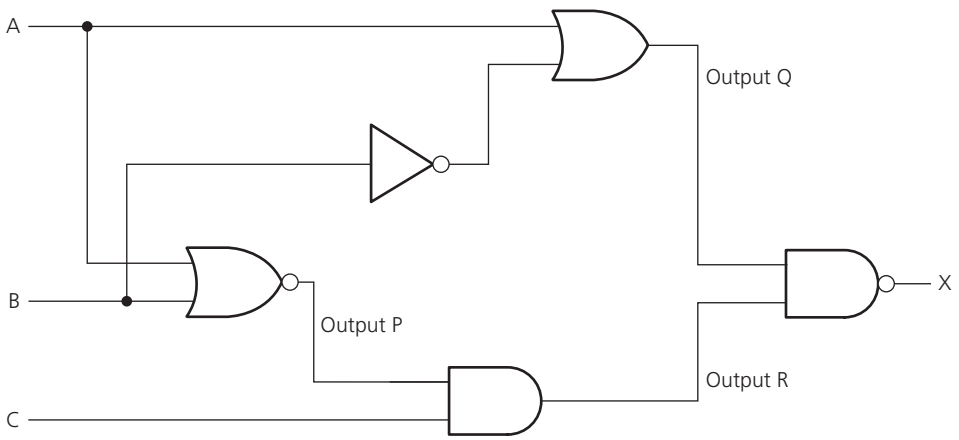
Input values			Working space	Output X
A	B	C		
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

First, we need to break down the logic statement. Assign:

i P = (A NOR B)
ii Q = (A OR NOT B)
iii R = (P AND C)

And then note that X = R NAND Q

Now draw the logic gates for statements i to iii and connect them together.



▲ **Figure 10.33**

We can then fill out the truth table in stages, starting with P, Q and then R, followed finally by X:

Input Values			Working Space			Output
A	B	C	P = (A NOR B)	Q = (A OR NOT B)	R = (P AND C)	X
0	0	0	1	1	0	1
0	0	1	1	1	1	0
0	1	0	0	0	0	1
0	1	1	0	0	0	1
1	0	0	0	1	0	1
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1

Find out more

Boolean algebra appears throughout this chapter and is the official method for depicting logic statements. Try using truth tables to prove the following pairs of Boolean (logic) statements are the same:

i $A + (A \cdot B) = A$
ii $A + \bar{A} \cdot B = A + B$
iii $(A + B) \cdot (A + C) = A + B \cdot C$
iv $\overline{(A \cdot B)} = \bar{A} + \bar{B}$
v $A + B = \bar{A} + \bar{B}$

Activity 10.4

1 Draw the logic circuits and complete the truth tables for the following logic or Boolean expressions:

a X = (A OR B) OR (NOT A AND B)
b Y = (NOT A AND NOT B) AND (NOT B OR C)
c T = 1 if (switch K is ON or switch L is ON) OR (switch K is ON and switch M is OFF) OR (switch M is ON)
d X = (A AND NOT B) OR (NOT B AND C)
e R = 1 if (switch A is ON and switch B is ON) AND (switch B is ON or switch C is OFF)

2 Produce the logic circuit and complete a truth table to represent the following scenario.

A chemical process is protected by a logic circuit. There are three inputs to the logic circuit representing key parameters in the chemical process. An alarm, X, will give an output value of 1 depending on certain conditions in the chemical process. The following table describes the process conditions being monitored:

Parameter description	Parameter	Binary value	Description of condition
chemical reaction rate	R	0	reaction rate < 40 mol/l/sec
		1	reaction rate >= 40 mol/l/sec
process temperature	T	0	temperature > 115°C
		1	temperature <= 115°C
concentration of chemicals	C	0	concentration <= 4 mol
		1	concentration > 4 mol

An alarm, X, will generate the value 1 if:

either	reaction rate < 40 mol/l/sec
or	concentration > 4 mol AND temperature > 115°C
or	reaction rate >= 40 mol/l/sec AND temperature > 115°C

3 Produce the logic circuit and complete a truth table to represent the following scenario.

A power station has a safety system controlled by a logic circuit. Three inputs to the logic circuit determine whether the output, S, is 1. When S = 1 the power station shuts down.

The following table describes the conditions being monitored.

Parameter description	Parameter	Binary value	Description of condition
gas temperature	G	0	gas temperature <= 160°C
		1	gas temperature > 160°C
reactor pressure	R	0	reactor pressure <= 10 bar
		1	reactor pressure > 10 bar
water temperature	W	0	water temperature <= 120°C
		1	water temperature > 120°C

Output, S, will generate a value of 1, if:

either gas temperature > 160°C AND water temperature <= 120°C

or gas temperature <= 160°C AND reactor pressure > 10 bar

or water temperature > 120°C AND reactor pressure > 10 bar

4 A car's engine management system uses three sensors A, B and C. The data from these sensors forms the input to a logic circuit. When the output (X) from the logic circuit is 1 a signal is sent to a warning light on the dash board of the car.

The following table describes the conditions being monitored.

Parameter description	Parameter	Binary value	Description of condition
exhaust temperature	E	0	exhaust temperature > 400°C
		1	exhaust temperature <= 400°C
oil pressure	P	0	oil pressure <= 10 bar
		1	oil pressure > 10 bar
water temperature	W	0	water temperature <= 110°C
		1	water temperature > 110°C

Output, X, will generate a value of 1, if:

either exhaust temperature > 400°C AND oil pressure > 10 bar

or oil pressure <= 10 bar AND water temperature > 110 °C

or oil pressure > 10 bar AND water temperature > 110 °C

Produce:

i a truth table,

ii a logic expression and

iii a logic circuit

to represent the above scenario.

Also confirm that the output from your logic circuit matches the output from your truth table in part **i**.

5 The following truth table is for a logic gate called the ***XNOR*** gate.

By completing the truth table below, show that the XNOR gate can be represented by the following logic expression:

(A AND B) OR (NOT A AND NOT B)

A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

A	B	(A AND B)	(NOT A AND NOT B)	(A AND B) OR (NOT A AND NOT B)
0	0			
0	1			
1	0			
1	1			

Advice

Regarding question 5, XNOR gates are not on the syllabus

Extension

The following two exercises are designed to help students thinking of furthering their study in Computer Science at A Level standard. The two topics here are not on the syllabus and merely show how some of the topics in this chapter can be extended to this next level. The two topics extend uses of logic circuits in the real world and the use of full adders and half adders.

Topic 1: Logic circuits in the real world

Anybody reading this chapter with an electronics background will be aware that the design of logic circuits is considerably more complex than has been described.

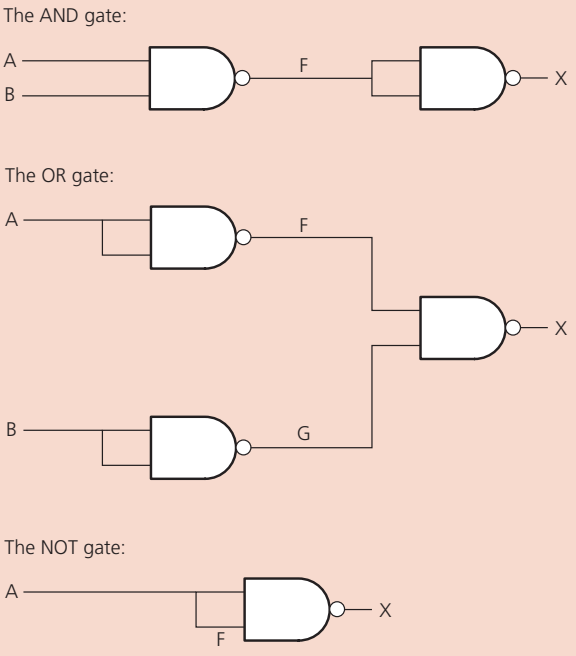
This chapter has described in detail some of the fundamental theories used in logic circuit design. This will give the reader sufficient grounding to cover all existing IGCSE and O Level syllabuses. However, it is worth finally discussing some of the other aspects of logic circuit design that will interest any student considering the A Level course.

Electronics companies need to consider the cost of components, ease of fabrication and time constraints when designing and building logic circuits. We will outline two possible ways electronics companies can standardise logic circuit design:

- One method is to use 'off-the-shelf' logic units and build up the logic circuit as a number of 'building blocks'
- Another method involves simplifying the logic circuit as far as possible; this may be necessary where room is at a premium (for example, in building circuit boards for use in satellites to allow space exploration).

Using logic 'building blocks'

One very common 'building block' is the NAND gate. It is possible to build up any logic gate, and therefore any logic circuit, by simply linking together a number of NAND gates. For example, the AND, OR and NOT gates can be built from NAND gates as shown here in Figure 10.34:



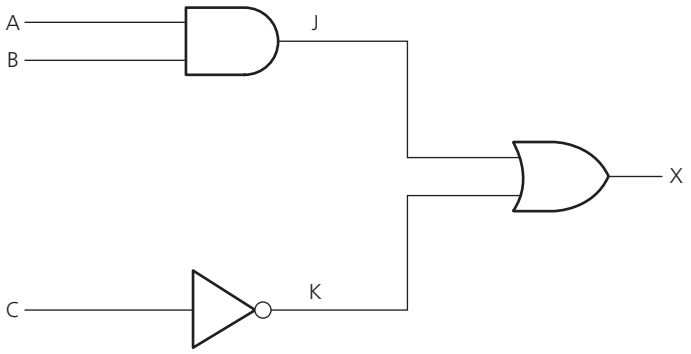
▲ **Figure 10.34**

Task 1

By drawing the truth tables, show that the three circuits in Figure 10.34 can be used to represent AND, OR and NOT gates.

Task 2

1 Show how the following logic circuit could be built using NAND gates only. Also complete truth tables for both logic circuits to show that they produce identical outputs.

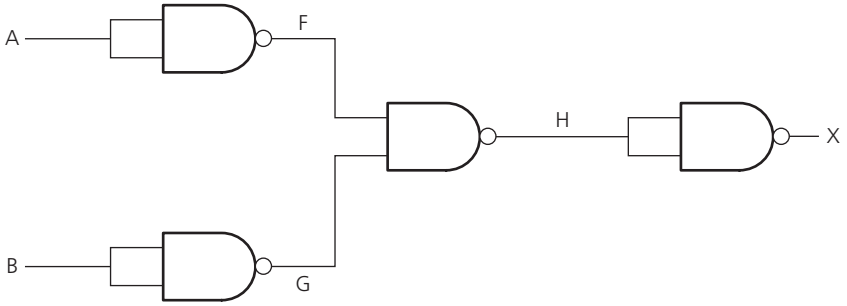


▲ **Figure 10.35**

2 Show how the XOR gate could be built from NAND gates only. Complete a truth table for your final design to show it that it produces the same output as a single XOR gate.

Task 3

By drawing a truth table, which single logic gate has the same function as the following logic circuit made up of NAND gates only?



▲ **Figure 10.36**

Topic 2: Half adder and full adder circuits

In Chapter 10, the use of logic gates to create logic circuits to carry out specific tasks was discussed in much detail. Two important logic circuits used in computers are:

- the **half adder circuit**
- the **full adder circuit**.

Half adder

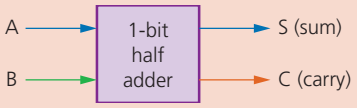
One of the basic operations in any computer is binary addition. The half adder circuit is the simplest circuit; this carries binary addition on 2 bits generating two outputs:

- the sum bit (S)
- the carry bit (C).

If you consider 1 + 1 this will give the result 1 0 (denary value 2). The '1' is the carry and '0' the sum. As a truth table this is:

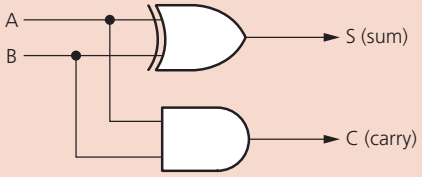
INPUT		OUTPUT	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

This is often shown in graphic form as:



▲ **Figure 10.37**

Or as a logic circuit:



▲ **Figure 10.38**

As you have probably guessed already, the half adder is unable to deal with the addition of more than two bits (for example, an 8-bit byte). To enable this, we have to consider the full adder circuit.

Full adder

Consider the following binary sum using 5-bit numbers:

A	0	1	[1]	1	0
B	0	0	[0]	1	1
S	1	0	[0]	0	1
C	1	1	[1]		

← This is the sum produced from the addition

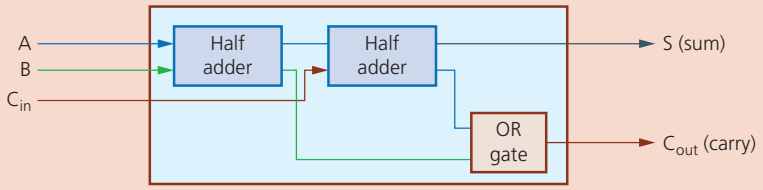
← This is the carry from the previous bit position

The sum shows how we have to deal with CARRY from the previous column. There are three inputs to consider in this third column, for example, A = 1, B = 0 and C = 1 (S = 0). This is why we need to join two half adders together to form a full adder, as shown in Figure 10.40

▲ **Figure 10.39**

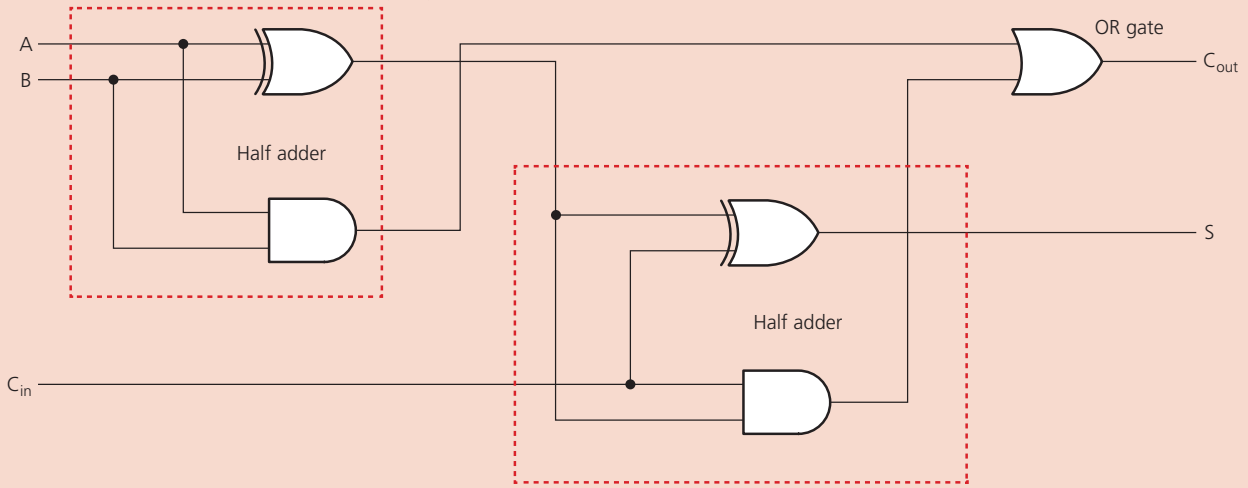
Link

See Section 1.1 for more details of binary addition.



▲ **Figure 10.40**

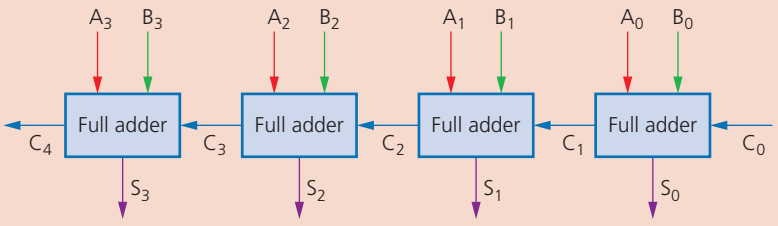
This has an equivalent logic circuit; there are a number of ways of doing this. For example, the following logic circuit uses OR, AND and XOR logic gates:



▲ **Figure 10.41**

Truth table for the full adder circuit:

INPUT			OUTPUT	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



▲ **Figure 10.42**

As with the half adder circuits, different logic gates can be used to produce the full adder circuit.

The full adder is the basic building block for multiple binary additions. For example, the following diagram shows how two 4-bit numbers can be summed using four full adder circuits:

In this chapter, you have learnt about:

- ✔ recognise the functions, symbols and truth tables for the logic gates: NOT, AND, OR, NAND, NOR and XOR
- ✔ create logic circuits from a given problem, logic expression or truth table
- ✔ complete a truth table from a given problem, logic (or Boolean) expression or logic circuit
- ✔ write a logic (or Boolean) expression from a given problem, logic circuit or truth table

Key terms used throughout this chapter

logic gate – an electronic circuit that relies on 'on/off' logic; the most common gates are NOT, AND, OR, NAND, NOR and XOR

logic circuit – these are formed from a combination of logic gates and designed to carry out a particular task; the output from a logic circuit will be 0 or 1

truth table – a method of checking the output from a logic circuit; a truth table lists all the possible binary input combinations and their associated outputs; the number of outputs will depend on the number of inputs; for example, two inputs have 2^2 (4) possible binary combinations, three inputs have 2^3 (8) possible binary combinations, and so on

Boolean algebra – a form of algebra linked to logic circuits and based on TRUE or FALSE

Exam-style questions

1 a Produce:

i a Boolean or logic expression

ii a logic circuit

for the truth table shown:

A	B	X
0	0	0
0	1	1
1	0	0
1	1	0

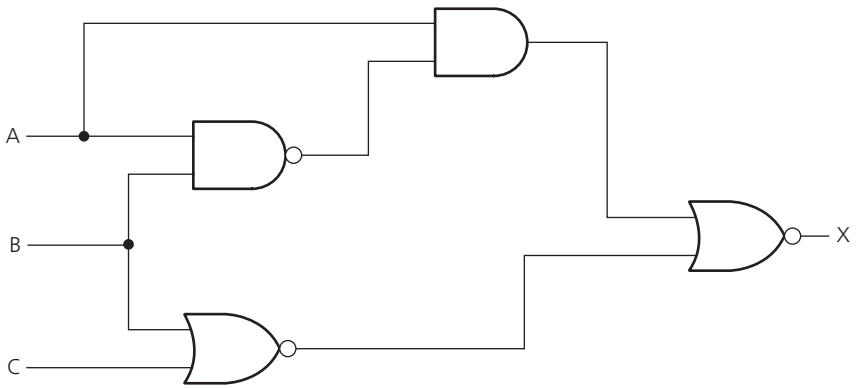
[4]

b Produce:

i a logic expression

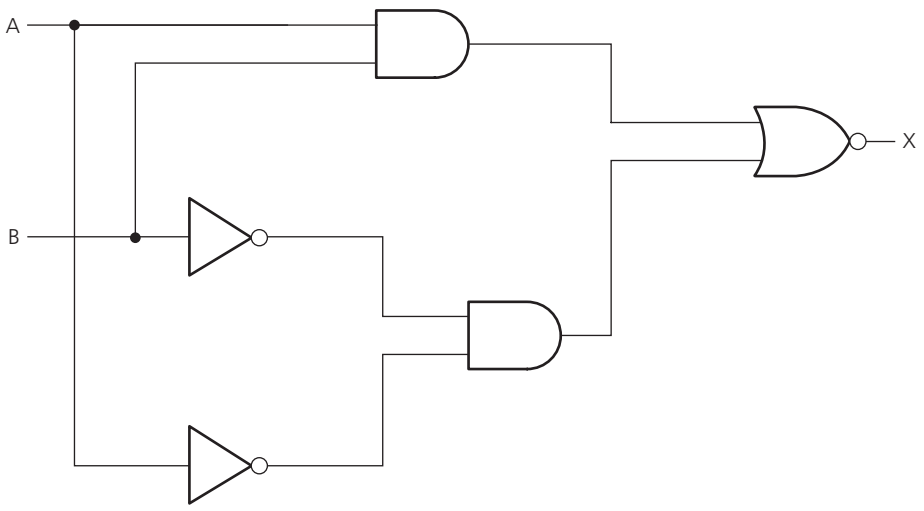
ii a truth table

for the logic circuit shown below:



[6]

2 a Write a logic expression for the following logic circuit:

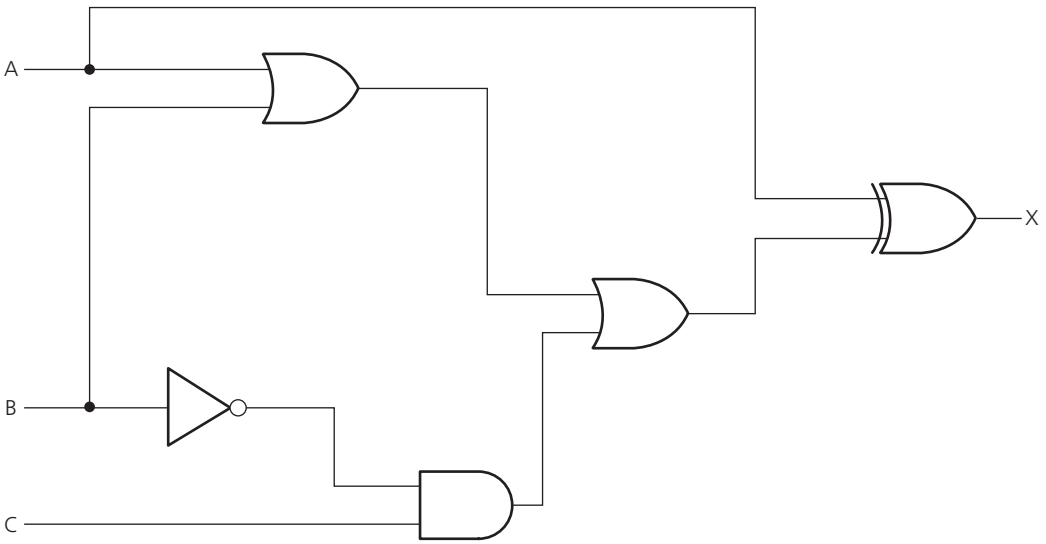


[3]

b Draw a logic circuit which represents the following Boolean expression:

X = NOT (A AND B) OR (A AND NOT B) [3]

c Complete the truth table for the following logic circuit:



[4]

3 A motor driving a water pump has a safety system which uses three inputs to a logic circuit. A certain combination of conditions results in an output, X, from the logic circuit being equal to 1. When the value of X = 1 then the motor and pump are shut down.
The following table shows which parameters are being monitored and form the three inputs to the logic circuit.

Parameter description	Parameter	Binary value	Description of condition
motor speed	S	0	motor speed <= 2000 rpm
		1	motor speed > 2000 rpm
bearing temperature	T	0	bearing temperature <= 90°C
		1	bearing temperature > 90°C
water velocity	V	0	water velocity <= 5 m/s
		1	wind velocity > 5 m/s

The output, X, will have a value of 1 if any of the following combination of conditions occur:

- **either** motor speed > 2000 rpm and bearing temperature > 90°C
- **or** motor speed <= 2000 rpm and water velocity <= 5 m/s
- **or** bearing temperature > 90°C and water velocity <= 5 m/s

a Design a logic circuit for the above scenario. [7]

b Complete a truth table for the above scenario. [4]

4 a Write a logic expression for the truth table below.

X	Y	Z
0	0	1
0	1	0
1	0	1
1	1	0

b Draw a logic circuit from the above truth table. [4]

5 A factory manufactures plastic pipes. It uses logic circuits to control the manufacturing process.

a Consider the logic gate:



Complete the truth table for this logic gate.

Input A	Input B	Output
0	0	
0	1	
1	0	
1	1	

[1]

b Consider the truth table:

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	0

State the **single** logic gate that produces the given output. [1]

c Plastic pipes of various sizes are manufactured by heating the plastic and using pressure. The manufacturing system uses sensors to measure the pressure (P), temperature (T) and speed (S) of production.
The inputs to the manufacturing system are:

Input	Binary value	Condition
P	1	pressure is > 5 bar
	0	pressure is <= 5 bar
T	1	temperature is > 200 degrees Celsius
	0	temperature is <= 200 degrees Celsius
S	1	speed is > 1 metre per second
	0	speed is <= 1 metre per second

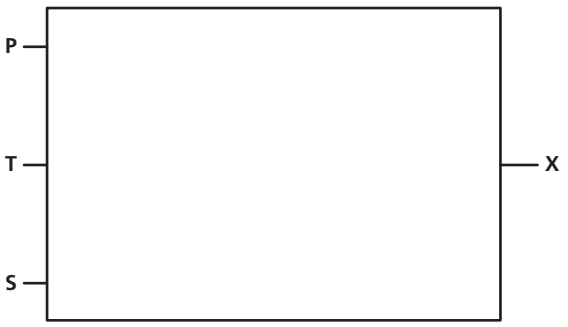
The system will sound an alarm (**X**) when certain conditions are detected. The alarm will sound when:

temperature is > 200 degrees Celsius and the pressure is <= 5 bar

or

speed is > 1 metre per second and the temperature <= 200 degrees Celsius

Draw a logic circuit to represent the above alarm system.
Logic gates used must have a maximum of **two** inputs. [5]



d Give **two** benefits of using sensors to monitor the manufacture of the plastic pipes. [2]

Cambridge IGCSE Computer Science (0478) Paper 11 Q7, Oct 2019

6 A DVD recorder is protected by three sensors (S, T and P). The output from these sensors forms the inputs to a logic circuit. A certain combination of input values produces an output of 1 from the logic circuit. When this occurs a warning message is shown on the DVD display. The following table shows which parameters are being monitored and form the three inputs to the logic circuit:

Parameter description	Parameter	Binary value	Description of condition
rotation speed	S	0	rotation speed >= 1000 rpm
		1	rotation speed < 1000 rpm
tilt angle	T	0	tilt angle >= 30°
		1	tilt angle < 30°
laser power	P	0	laser power < 150 mW
		1	laser power >= 150 mW

The output X will have a value of 1 if any of the following combination of conditions occurs:

- either rotation speed >= 1000 rpm and tilt angle < 30°
- or rotation speed < 1000 rpm and laser power >= 150 mW
- or tilt angle >= 30° and laser power < 150 mW

a Write down a logic or Boolean expression for the above system. [2]

b Draw a logic circuit to monitor the above system. [6]

c Complete the truth table for the above system:

Input values			Working space	Output X
S	T	P		
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

[4]